DocChat

On-device PDF question answering — how it works and what it does not do yet

> **Status (branch llms-and-chat):** upload, indexing and question answering run entirely on the phone. Login and a custom model-download server are deliberately not built yet; models download from Hugging Face. Everything has been verified on a Windows laptop through automated tests; **nothing has been tested on a real phone yet.**

1. Goal

Answer questions about the user's PDFs without sending documents, questions or answers to any server. The app follows the October 2026 architecture guide: on-device retrieval-augmented generation (RAG). A small embedding model indexes each PDF once; for every question the app finds the relevant passages and a small language model writes the answer from them, citing pages.

Key principle: the answering model never sees the whole document. Retrieval decides which few passages it reads, so **retrieval quality matters more than model size**.

2. Models and the download flow

Shown to users as	Model (never shown in the app)	Size	Role
Default resources pack	EmbeddingGemma-300M, Q8_0	334 MB	Always downloaded. Indexes PDFs and searches them (768-dimension vectors).
High accuracy	Qwen3-4B, Q4_K_M	2.5 GB	Answering model for 8 GB+ phones. Best accuracy on multi-section and numeric questions.
Basic	Qwen3-1.7B, Q5_K_M	1.26 GB	Answering model for 6 GB phones. Faster; weaker on questions that span sections.

- **Recommendation:** the app reads the phone's RAM and pre-selects High accuracy when it reports 7 GB or more (8 GB phones report about 7.2–7.6 GB); otherwise Basic.
- **First launch:** the resources pack starts downloading immediately. The user picks an answering model and taps a single **Download** button. Once a download starts, the choice is locked: the other model, the Wi-Fi switch and the button are disabled until it finishes or fails. The app opens only once both downloads have finished and passed a SHA-256 check.
- **Downloads** use the operating system's background transfer service, so they continue with the app closed, resume after network drops, show progress in a notification, and default to Wi-Fi only. The app never starts a second copy of a download, progress never goes backwards, and the in-app banner shows one combined percentage for everything downloading.
- **Changing models later** keeps the current model in use until the new one is ready.

3. Indexing a PDF (once, on upload)

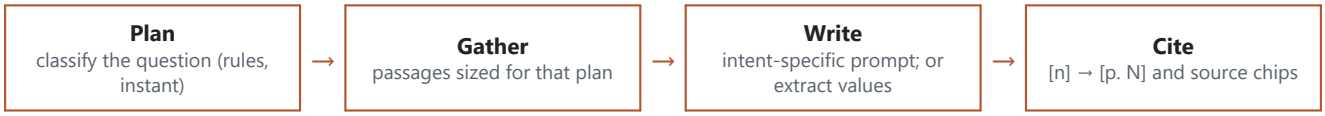


- **Chunks** are about 1,600 characters with 200 characters of overlap and never cross a page, so every passage can be cited by page. A chunk over 1,024 tokens is split again.
- **Stored per document:** page text (for the viewer), chunk text with its vector, a full-text keyword index (SQLite FTS5), and **extracted facts** — dates, amounts, durations, percentages and reference IDs with the sentence they

appear in, found by pattern matching.

- **Background:** uploads are queued and indexed one at a time. On Android a foreground-service notification shows a progress bar and keeps indexing alive after the user leaves the app.
- **Deleting a document** removes its file, pages, chunks, vectors, keyword entries and facts.

4. Answering a question



4.1 Query planning

Before anything is retrieved, a rule-based planner decides what kind of question it is. Rules were chosen over asking the model because they are instant and predictable on a phone. Anything not recognised is treated as a normal lookup — the behaviour every question had before planning existed — so a missed classification never does worse than the baseline.

Intent	Example	How passages are gathered
Lookup (default)	What is the hotel cap in Mumbai?	Hybrid search, top 5 passages
Follow-up	…and in other cities? · what about Pune?	Searched together with the chat's topic question; the topic and the previous exchange are given to the model (see 4.4)
Section summary	Summarize the leave policy	Best-matching passage plus its neighbours
Document summary	Summarize this document · Key points	Passages sampled evenly from first to last page (up to the 3 newest documents)
List	List all the deadlines	About 12 short excerpts (matching sentences only) plus extracted facts of the asked kind
Compare	Compare earned leave and sick leave	One search per side, passages labelled by side
Calculation	Total of … · How many days until …	Passages plus facts; the model extracts values as JSON and the app does the arithmetic
Small talk	thanks · hi	None — instant reply without the model

- **Document named in the question:** in an "All documents" chat, a question such as "In the lease, …" searches only the matching document.
- **How much is read:** lookups and follow-ups read the top 5 passages whole (up to about 8,000 characters, ~2,000 tokens), so the sentence holding the answer is never cut off. Every other intent is held to about 7,000 characters, spent differently: summaries and comparisons trim each passage, lists read many short excerpts. Broad questions therefore read no more than a lookup, which keeps answer time predictable on a phone.

4.2 Hybrid search

Two searches run side by side and their rankings are merged with Reciprocal Rank Fusion: **vector search** (meaning — cosine similarity over every stored vector) and **keyword search** (exact terms such as "EXP-12", BM25 ranking in FTS5). Each returns 20 candidates. There is no reranker (see section 6).

4.3 Writing the answer

- Each intent has its own prompt and output length (from about 300 tokens for calculations to 600 for lists). All prompts require answering only from the passages, citing them, quoting values exactly, respecting qualifiers such as "during" vs "after" probation, and replying "Not found in your documents." when the answer is not there.
- **Calculations are never done by the model.** It returns the values as constrained JSON with greedy decoding; the app computes totals, averages, differences, day counts and counts. If extraction fails, the answer lists the

values and says the calculation could not be done.

- Answers stream into the chat with a Stop button; partial answers are saved every 3 seconds.
- **In the background:** Android keeps answering through the foreground service and posts "Your answer is ready"; the model is unloaded from memory afterwards. iOS stops cleanly (it forbids background GPU work) and keeps the partial answer with a note to regenerate.

4.4 Context per chat

Each chat keeps its own messages, title and scope (one document or "All documents") on the device, and is treated as one consistent conversation:

- **Scope is locked once a chat has messages.** Choosing a different document in the scope sheet starts a new chat about it instead of changing the current one, so earlier answers and follow-ups never mix contexts.
- **Topic memory.** The app replays the chat's earlier questions to find the topic question — the last question that was not a follow-up — skipping small talk such as "thanks". Every follow-up is searched together with that topic, so a chain like "hotel cap in metro cities?" → "and in other cities?" → "what about Pune?" stays on hotel caps. A new self-contained question starts a new topic.
- Nothing extra is stored and no model call is added: the context is recomputed from the saved messages, so it also survives restarts. The model sees the topic question and the last exchange, not the whole chat.

5. Evaluation

An end-to-end evaluation (`test/eval/answer_eval_test.dart`, opt-in) indexes the sample employee handbook and the architecture guide together and runs every intent through planning, retrieval and the High accuracy model on the laptop. It took four rounds of fixes; the final run passed all 16 cases, including a two-step follow-up ("what about Pune?") that stays on the hotel-cap topic. That answer is only partly right: it gives both caps but does not infer that Pune is not one of the listed metro cities.

Intent	Question	Answer	Laptop time
Lookup	Earned leave days carried forward	10 days	10 s
Follow-up	"and in other cities?" after the metro hotel cap	INR 5,000	10 s
Follow-up chain	"what about Pune?" two follow-ups later	Partly: stays on topic, gives both caps, does not infer Pune is non-metro	16 s
Lookup	Notice period after / during probation	60 days / 15 days	5–7 s
Section summary	Summarize the leave policy	4 correct bullets	11 s
Document summary	This document / all my documents	Correct, 3 bullets per document	20 s
List	All the deadlines	7 complete items	20 s
Compare	Earned vs sick leave	Correct differences	12 s
Calculation	Metro hotel cap + daily allowance	INR 9,500 (computed)	15 s
Calculation	Metro vs other-city hotel cap	INR 2,000 (computed)	15 s
Calculation	Effective date to 31 Dec 2026	274 days (computed)	16 s
Small talk	thanks	Instant reply	0 s
Not in documents	Policy on pets	Not found in your documents.	7 s
Lookup (2nd PDF)	Model for 6 GB phones	Qwen3-1.7B	11 s

Fixes found by the evaluation: qualifier mix-ups ("after" vs "during" probation), comparisons that invented facts to fill a requested bullet count, list items cut off by PDF line breaks, unreliable value extraction (and one case where the model did the arithmetic itself and got 238 instead of 274 days), and over-long multi-document summaries.

55 unit tests cover the chunker, keyword queries, fact extraction, the planner (including follow-up chains), the calculator, the chat scope lock and the setup screen (including the download lock) and download progress.

> Phone timings will be several times the laptop's: the guide estimates 30–90 seconds before the first word for a 4B model on a phone CPU. This has not been measured yet.

6. Key decisions

- **No reranker.** The guide's MiniLM reranker could not be used: every published GGUF conversion is broken, because llama.cpp's converter drops the pooler layer its classifier needs (scores were all ≈ 0). jina-reranker tiny was weak (3 of 5 correct). The official Qwen3-Reranker-0.6B was accurate but 639 MB, about 10 s per question on a phone, and did not beat hybrid search on a 10-page test (7/8 vs 6/8 correct top results). Decision: ship without a reranker and revisit with long documents.
- **Rules, not the model, classify questions** — no added latency, predictable, safe fallback.
- **Arithmetic in code**, following the guide — small models get sums and date differences wrong.
- **A reading budget for broad questions** (about 7,000 characters) so lists, summaries and comparisons are no slower than lookups; lookups keep their top 5 passages whole so answers are never cut off.
- **Model names hidden from users**; only "High accuracy", "Basic" and "Default resources pack" appear.
- **Encryption deferred.** It would not slow the model: model files are public and stay unencrypted, and database encryption costs milliseconds per query.

7. Limitations

Ordered by impact. "High" items should be resolved before real users rely on the app.

Area	Limitation	How to address it	Impact
Testing	**Never run on a real phone** All verification so far is on a Windows laptop. Speed, memory, downloads, notifications and RAM detection are unverified on devices; no iOS build has been made.	Install the APK on 6 GB and 8 GB Android phones; build on a Mac for iOS.	**High**
Speed	**Slow answers with High accuracy** Android runs on the CPU only (GPU/Vulkan support is experimental). Estimated 30–90 s before the first word on a phone.	Measure on devices; consider Basic as default for borderline phones; test Vulkan as an opt-in.	**High**
Security	**No encryption at rest** The database (text, vectors, chats) and copied PDFs are plain files in app storage, protected only by device encryption. The guide requires app-level encryption.	SQLCipher with the key in Android Keystore / iOS Keychain; encrypt stored PDFs.	**High**
Retrieval	**No reranker** Ordering relies on fusing vector and keyword ranks. On long documents the right passage may rank below the top 5.	Convert MiniLM with the pooler kept, or measure Qwen3-Reranker on long PDFs.	**Medium**
Retrieval	**Chunks stop at page boundaries** A sentence or table that continues on the next page is split; short pages become tiny chunks.	Allow chunks across pages and store a page range for citations.	**Medium**
Retrieval	**Tables and scanned PDFs** Plain text extraction scrambles tables. Scanned PDFs fail with "No text found" (no OCR).	Table-aware extraction; on-device OCR (e.g. ML Kit).	**Medium**
Answers	**Thin summaries of long documents** Summaries read about 6 sampled passages, which is shallow for 100+ page documents.	Cache section-by-section summaries built once in the background.	**Medium**
Answers	**Long lists may be incomplete** Lists read about 12 excerpts plus extracted facts within the budget, with no warning when items may be missing.	Detect when the budget is exhausted and say so; page through results.	**Medium**
Answers	**Small-model mistakes remain** Occasional contradictions in comparisons and mixed-up qualifiers, reduced but not eliminated by prompts.	Larger evaluation set; reranker; stricter per-intent checks.	**Medium**
Distribution	**Models come from Hugging Face** URLs and checksums are built into the app; networks that block Hugging Face cannot download; changing a model needs an app release. The Basic model comes from a community repository (unsloth).	Own manifest and CDN with signed URLs after login (guide section 12).	**Medium**
Planner	**English, rule-based classification** Unusual phrasings and non-English questions fall back to a plain lookup. Fact patterns cover English and Indian formats (day-first dates).	Embedding-based intent matching as a second signal; more patterns.	**Low**
Conversation	**Follow-up memory is one topic deep** Follow-up detection is heuristic (short questions, pronouns, "and…"), and the model sees only the topic question and the last exchange — not a summary of the whole conversation.	Add a short rolling conversation summary if users need longer memory.	**Low**
Calculation	**Limited arithmetic** Mixed currencies are totalled separately (no conversion); dates without a year cannot be used.	Unit conversion table; ask the user for the missing year.	**Low**

Area	Limitation	How to address it	Impact
Platform	**Background limits** iOS pauses indexing and stops answers in the background. Android 15 limits this type of foreground service to about 6 hours a day; notifications need permission.	Resume interrupted work automatically on return.	**Low**
Resources	**Memory and storage** High accuracy plus the resources pack peak at about 3.4 GB RAM; the OS may kill the app in the background. Vector search scans every vector (fine to about 10,000 chunks).	Unload models more aggressively; sqlite-vec index for large libraries.	**Low**
Data	**No re-index on model change, no backup** Changing the embedding model would silently hide existing documents. Uninstalling deletes all documents and chats.	Version the index and re-index in the background; optional encrypted export.	**Low**
Product	**Login and account features** Login is not implemented; "Sign out" and "Delete account" in the profile menu are placeholders. Chat answers are plain text (no formatting).	Implement the login backend; render basic formatting.	**Low**
Licensing	**Model licences need review** EmbeddingGemma is under Gemma terms; Qwen3 is Apache 2.0.	Legal review before commercial release.	**Low**

8. Suggested next steps

- Install the current APK on a 6 GB and an 8 GB Android phone; time indexing and answers for each model.
- Add encryption at rest (SQLCipher plus Keystore/Keychain) before handling real documents.
- Grow the evaluation to 50–100 questions on long, real documents; use it to decide on a reranker and on cross-page chunking.
- Move model downloads to your own manifest and CDN once login exists.